

Universidad Abierta Interamericana



Alumnos: Ignacio Carignano

Carrera: Ingeniería en Sistemas Informáticos

Materia: Metodologías ágiles

Profesor: Alejandro Sartorio

Fecha de realización: 25/06/26

Sistema Web de Gestión de Inventario

Introducción y enfoque metodológico

El presente trabajo propone el análisis, diseño, construcción, pruebas, despliegue y mantenimiento de una aplicación web destinada a gestionar el inventario de una organización. La resolución se organiza de acuerdo con una secuencia de desarrollo en cascada: primero se establecen los requisitos; luego se elaboran los diseños; posteriormente se plantea la implementación, las pruebas, el despliegue y el mantenimiento.

El alcance definido para esta primera versión incluye la administración de usuarios, categorías, productos, movimientos de stock, alertas por stock bajo y reportes básicos. Se evita incluir funcionalidades que no sean necesarias para el objetivo principal, de modo de mantener un alcance coherente y verificable.

1. Análisis de requerimientos

Ejercicio 1. Requisitos funcionales y no funcionales

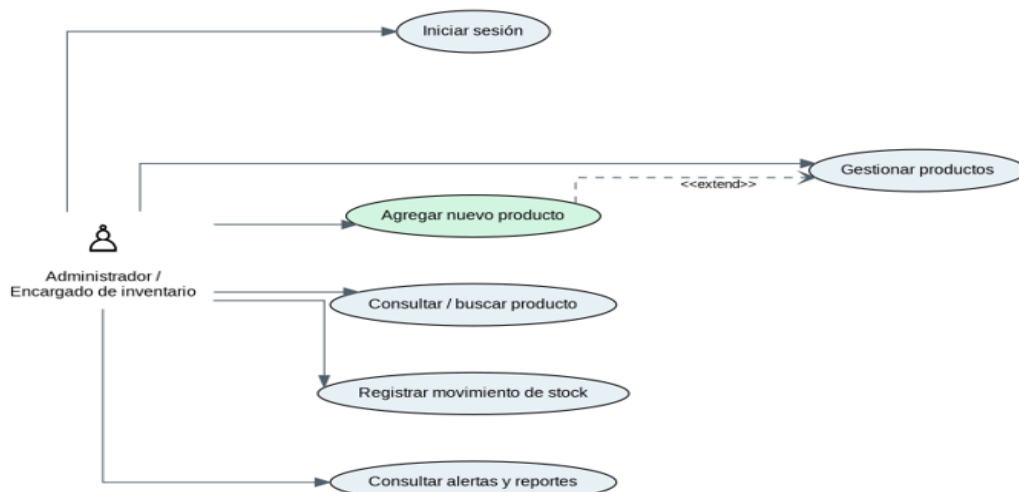
Enunciado: Un cliente solicita una aplicación web para gestionar su inventario. Se definen los siguientes requisitos.

Código	Requisito funcional
RF01 - Autenticación	El sistema debe permitir que los usuarios autorizados inicien y cierren sesión mediante correo electrónico y contraseña.
RF02 - Gestión de categorías	El sistema debe permitir registrar, consultar, modificar y desactivar categorías de productos.
RF03 - Alta de producto	El sistema debe permitir registrar productos indicando código, nombre, descripción, categoría, precio, stock inicial y stock mínimo.
RF04 - Consulta y búsqueda	El sistema debe permitir visualizar, buscar y filtrar productos por código, nombre o categoría.
RF05 - Edición y baja lógica	El sistema debe permitir modificar productos existentes y marcarlos como inactivos sin eliminar su historial.

RF06 - Movimientos de stock	El sistema debe permitir registrar entradas, salidas y ajustes, conservando la fecha, el usuario responsable y el motivo.
RF07 - Alertas	El sistema debe informar los productos cuyo stock actual sea menor o igual al stock mínimo establecido.
RF08 - Reportes	El sistema debe permitir consultar reportes básicos de existencias, movimientos recientes y productos con stock bajo.
RF09 - Seguridad por roles	El sistema debe limitar las operaciones de administración a usuarios con permisos adecuados.

Código	Requisito no funcional
RNF01 - Usabilidad	La interfaz debe ser clara e intuitiva para usuarios no técnicos.
RNF02 - Rendimiento	Las operaciones comunes de búsqueda, consulta y registro deben responder en un tiempo adecuado aun con muchos productos.
RNF03 - Seguridad	Las contraseñas deben almacenarse protegidas; el acceso debe requerir autenticación y validación de permisos.
RNF04 - Integridad	No se debe permitir que existan dos productos con el mismo código ni que se registren stocks o precios negativos.
RNF05 - Disponibilidad	La aplicación debe estar disponible durante el horario operativo y debe contar con recuperación ante fallas.
RNF06 - Compatibilidad	La aplicación debe funcionar correctamente en navegadores web actuales y en pantallas de escritorio y móviles.
RNF07 - Mantenibilidad	La solución debe separar presentación, lógica de negocio y acceso a datos para facilitar correcciones y ampliaciones.
RNF08 - Respaldo	La información crítica debe poder respaldarse y restaurarse mediante copias de seguridad periódicas.

Ejercicio 2. Caso de uso: Agregar un nuevo producto



Elemento	Especificación
Código	CU-01
Nombre	Agregar nuevo producto
Actor principal	Administrador o encargado de inventario
Objetivo	Registrar un producto nuevo y dejarlo disponible para futuras consultas y movimientos de stock.
Precondiciones	El usuario inició sesión, posee permisos de inventario y existen categorías activas.
Disparador	El usuario selecciona la opción “Añadir producto” desde el panel principal o el módulo Productos.
Postcondiciones	El producto queda persistido; se registra un movimiento de entrada por el stock inicial y, si corresponde, una alerta de stock bajo.
Reglas de negocio	El código debe ser único; nombre y categoría son obligatorios; precio, stock inicial y stock mínimo no pueden ser negativos.

Curso básico

1. El usuario ingresa a la aplicación e inicia sesión.
2. El sistema valida las credenciales y muestra el panel principal.
3. El usuario accede al módulo Productos.
4. El usuario selecciona “Añadir producto”.
5. El sistema muestra el formulario de alta.
6. El usuario ingresa código, nombre, descripción, categoría, precio, stock inicial y stock mínimo.
7. El usuario confirma la operación mediante el botón Guardar.
8. El sistema valida campos obligatorios, formato numérico, unicidad del código y existencia de la categoría.
9. El sistema crea el producto en la base de datos.
10. El sistema registra un movimiento de tipo Entrada por el stock inicial.
11. El sistema genera una alerta si el stock inicial es menor o igual al stock mínimo.
12. El sistema muestra un mensaje de éxito y actualiza el listado de productos.

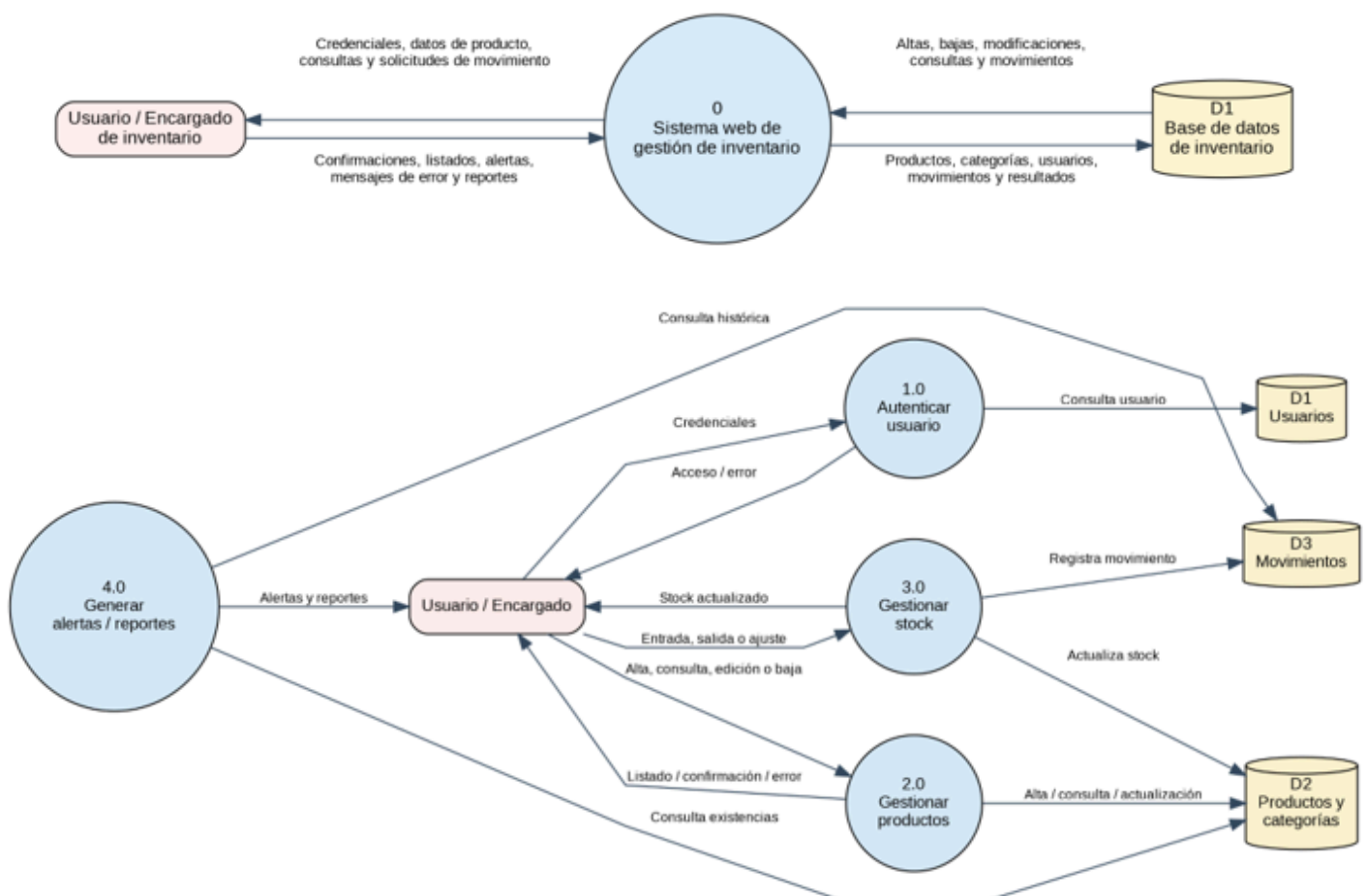
Flujos alternativos

- FA01 - Campos incompletos: el sistema indica los campos obligatorios pendientes y no realiza la persistencia.
- FA02 - Código duplicado: el sistema informa que el código ya se encuentra registrado y solicita uno diferente.
- FA03 - Valores inválidos: si precio o stocks son negativos, el sistema rechaza la operación e informa el error.
- FA04 - Categoría inexistente o inactiva: el sistema no permite asociar el producto y solicita seleccionar una categoría válida.
- FA05 - Usuario sin permiso: el sistema bloquea el acceso a la operación y muestra un aviso de autorización insuficiente.

2. Diseño del sistema

Ejercicio 3. Diagramas de flujo de datos

Se presenta un DFD de contexto y un DFD de nivel 1. El primero muestra la interacción global entre el usuario, el sistema y la base de datos. El segundo descompone las funciones principales del sistema.



Ejercicio 4. Diseño de interfaz de usuario para la pantalla Inicio

La pantalla Inicio se diseña como un panel operativo. Su finalidad es mostrar de forma inmediata el estado del inventario y ofrecer un acceso rápido al alta de productos.



- Barra lateral: permite navegar al inicio, productos, movimientos y configuración.
- Encabezado: identifica el sistema, el usuario y el estado general de conexión.
- Indicador principal: informa la cantidad total de unidades disponibles.
- Acceso rápido: abre el formulario para agregar un producto.
- Últimos movimientos: expone las operaciones de inventario más recientes.
- Alertas y categorías: permite detectar faltantes y conocer la distribución del stock.

3. Diseño del programa

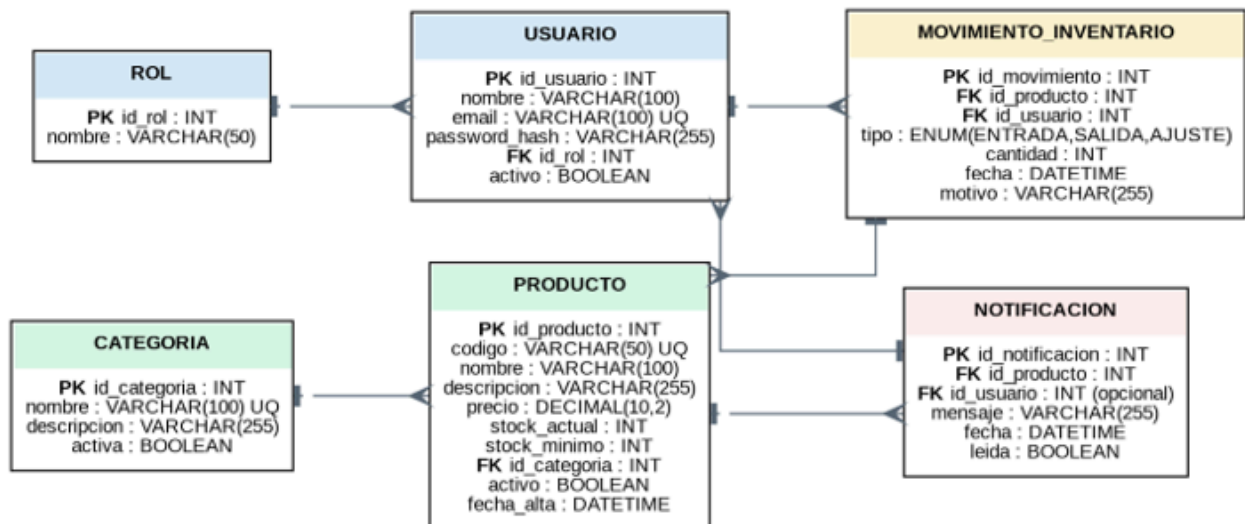
Ejercicio 5. Arquitectura propuesta y justificación



Se propone una arquitectura cliente-servidor con una separación lógica en capas. El frontend se ocupa de la interacción con el usuario y de validaciones de formato. El backend concentra las reglas de negocio y las validaciones definitivas. La capa de datos se responsabiliza de la comunicación con la base de datos relacional.

- Centralización de datos: todos los usuarios consultan una única fuente de verdad para conocer el stock real.
- Seguridad: la base de datos no queda expuesta al usuario final; las operaciones se ejecutan desde el backend con permisos controlados.
- Mantenibilidad: cambiar la interfaz no obliga a modificar las reglas de negocio ni la estructura de datos.
- Escalabilidad: pueden agregarse reportes, proveedores, nuevos roles o integraciones sin rehacer el sistema completo.
- Integridad: la lógica del servidor puede manejar transacciones para que el alta de un producto y su movimiento inicial se registren de manera consistente.

Ejercicio 6. Diseño de base de datos



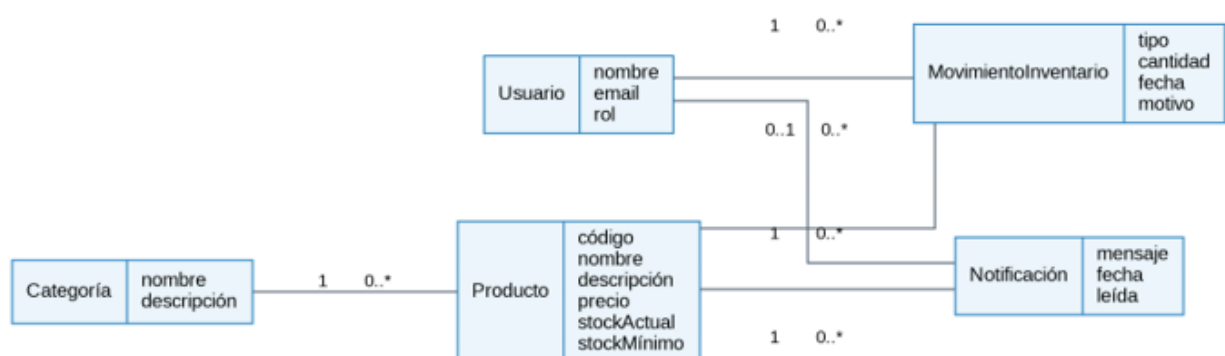
La base de datos propuesta es relacional. Las claves primarias identifican cada registro y las claves foráneas preservan la relación entre usuarios, categorías, productos, movimientos y notificaciones.

Regla	Descripción
Código único	El campo código de Producto debe tener una restricción UNIQUE.
Integridad numérica	Precio, stock_actual, stock_minimo y cantidad no pueden ser negativos.
Integridad referencial	Un producto debe pertenecer a una categoría existente; un movimiento debe referir a un producto y un usuario válidos.
Baja lógica	Los productos se marcan como inactivos cuando sea necesario, evitando eliminar el historial de movimientos.
Trazabilidad	Cada entrada, salida o ajuste debe generar un MovimientoInventario con fecha, usuario y motivo.

4. Diseño de la funcionalidad “Agregar un nuevo producto”

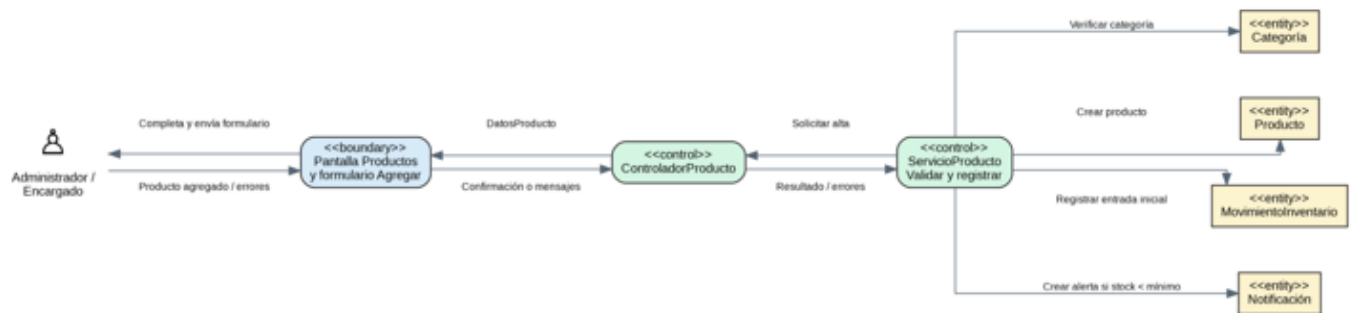
Los siguientes artefactos modelan el caso de uso CU-01. Se complementan entre sí: el dominio define conceptos; la robustez asigna responsabilidades; el prototipo plantea la interacción; la secuencia muestra mensajes; y el diagrama de clases prepara la implementación.

Diagrama de dominio



El modelo de dominio identifica las entidades del problema, sin depender todavía de decisiones técnicas de implementación. La entidad central es Producto, relacionada con Categoría y MovimientoInventario.

Diagrama de robustez



El diagrama de robustez separa tres tipos de elementos: boundary o frontera (pantallas y formularios), control (coordinación y validaciones) y entity o entidad (información persistente del dominio).

Prototipo de la funcionalidad

Nuevo producto

Completé la información para registrar el artículo.

CÓDIGO DEL PRODUCTO	NOMBRE DEL PRODUCTO
Ej: P-0001	Ej: Cemento Holcim
DESCRIPCIÓN	
Descripción breve del artículo	
CATEGORÍA	PRECIO
Seleccionar categoría	0,00
STOCK INICIAL	STOCK MÍNIMO
0	0

Cancelar
Guardar

El formulario contiene todos los datos necesarios para mantener consistencia con requisitos, lógica de negocio y base de datos: código, nombre, descripción, categoría, precio, stock inicial y stock mínimo.

Diagrama de secuencia

Diagrama de secuencia - Agregar nuevo producto

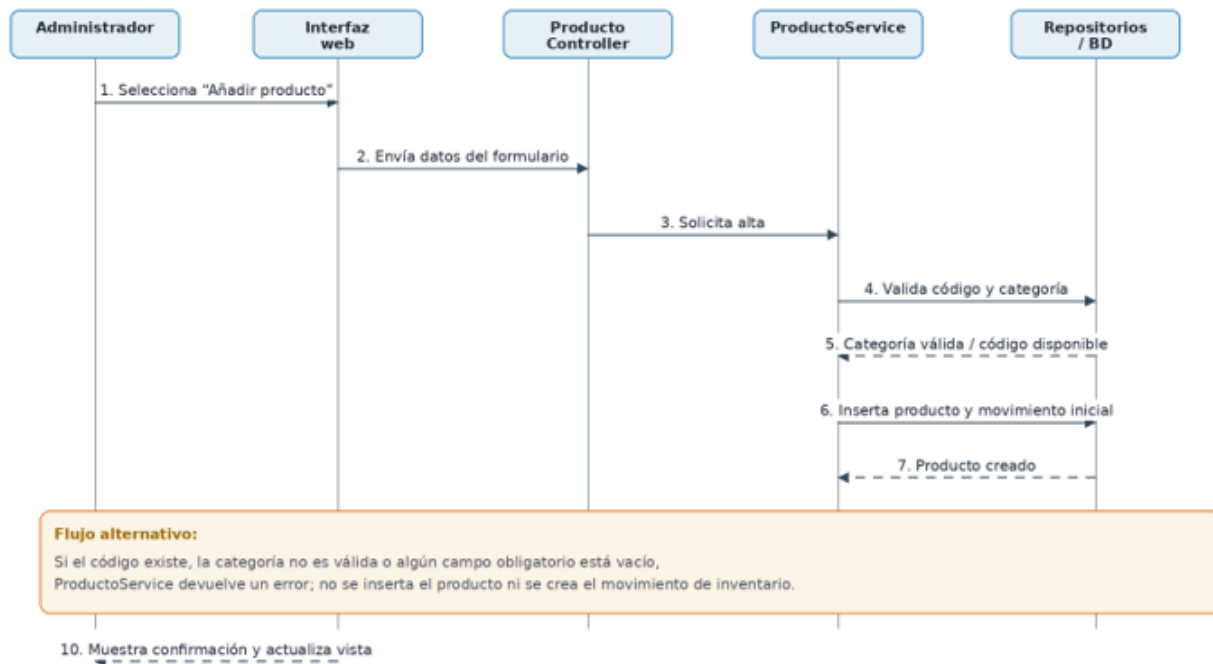
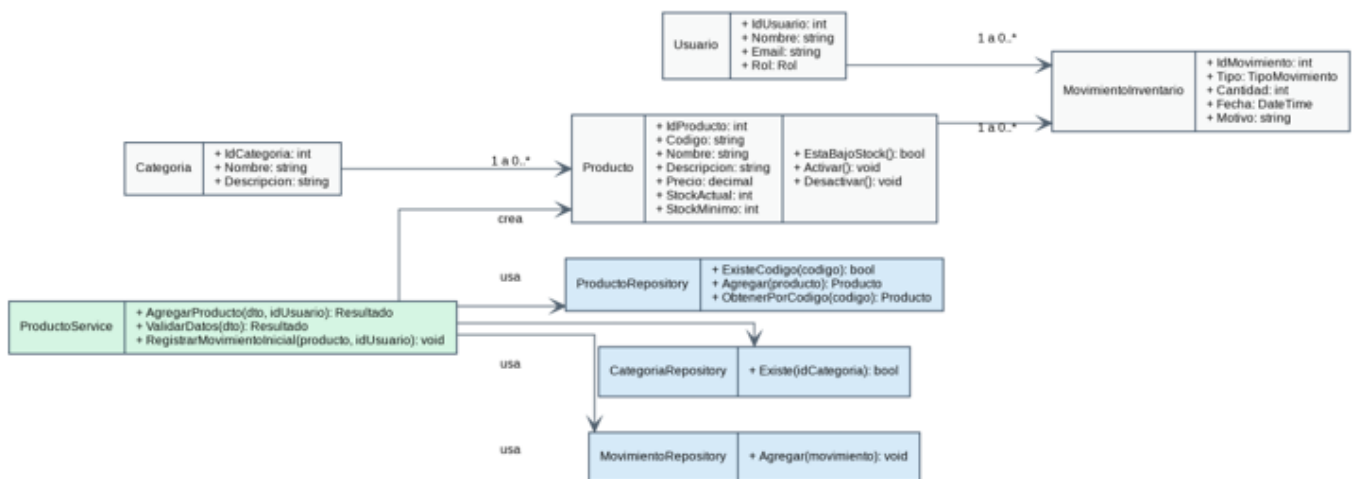


Diagrama de clases



Ejercicio 7. Implementación de la funcionalidad “Agregar un nuevo producto”

Para representar la implementación se desarrolló un prototipo frontend funcional en HTML, CSS y JavaScript. El formulario valida campos obligatorios, controla valores negativos y evita códigos duplicados. Los productos se almacenan localmente en el navegador

mediante localStorage, por lo que el prototipo puede demostrarse sin requerir un servidor real.

Archivo incluido en la entrega: inventario_prototipo.html. Al abrirlo en un navegador se puede registrar productos, visualizarlos en la tabla, actualizar el stock total y detectar productos con stock bajo.

Fragmento representativo de la validación de alta:

```
const productos = obtenerProductos();
if (!producto.codigo || !producto.nombre || !producto.categoria) {
    mostrarError("Completa los campos obligatorios.");
    return;
}
if (producto.precio < 0 || producto.stock < 0 || producto.stockMinimo < 0) {
    mostrarError("Precio y stocks no pueden ser negativos.");
    return;
}
if (productos.some(p => p.codigo.toLowerCase() === producto.codigo.toLowerCase())) {
    mostrarError("El código de producto ya existe.");
    return;
}
productos.push(producto);
guardarProductos(productos);
```

Ejercicio 8. Lógica de negocio de “Agregar un nuevo producto”

La lógica de negocio se ejecuta en el backend y debe ser independiente de la interfaz. Su finalidad es comprobar que la operación cumpla todas las reglas antes de modificar la información del inventario.

1. Verificar que el usuario autenticado posee permiso para gestionar inventario.
2. Recibir los datos capturados en el formulario de alta.
3. Validar que código, nombre y categoría estén presentes.
4. Validar que precio, stock inicial y stock mínimo tengan valores numéricos no negativos.

5. Consultar si ya existe un producto con el mismo código.
6. Verificar que la categoría exista y esté activa.
7. Crear y guardar el producto.
8. Registrar de forma asociada un MovimientoInventario de tipo Entrada por el stock inicial.
9. Comprobar si el stock inicial es menor o igual al stock mínimo; si lo es, generar una notificación.
10. Confirmar la operación al usuario y devolver los datos del nuevo producto.

Pseudocódigo de la operación

```
función AgregarProducto(datosProducto, idUsuario):  
    verificarPermiso(idUsuario, "GESTIONAR_INVENTARIO")  
    validarCamposObligatorios(datosProducto)  
    validarValoresNoNegativos(datosProducto.precio, datosProducto.stockInicial, datosProducto.stockMinimo)  
    si ProductoRepository.ExisteCodigo(datosProducto.codigo) entonces  
        devolver Error("El código de producto ya existe")  
    fin si  
    si no CategoriaRepository.ExisteActiva(datosProducto.idCategoria) entonces  
        devolver Error("La categoría seleccionada no es válida")  
    fin si  
  
    iniciarTransacción()  
    producto = ProductoRepository.Agregar(datosProducto)  
    MovimientoRepository.Agregar(Entrada(producto.id, datosProducto.stockInicial, idUsuario))  
    si producto.stockActual <= producto.stockMinimo entonces  
        NotificacionRepository.Agregar(AlertaStockBajo(producto.id))  
    fin si  
    confirmarTransacción()  
    devolver Éxito(producto)
```

5. Pruebas

Ejercicio 9. Pruebas unitarias

Las pruebas unitarias verifican reglas concretas de la lógica de negocio de manera aislada. La siguiente tabla plantea los casos mínimos que debe cubrir el servicio de productos.

ID	Caso	Entrada / condición	Resultado esperado
PU01	Alta válida	Datos completos, código único y categoría activa	Se crea el producto y se registra el movimiento inicial.
PU02	Nombre obligatorio	Nombre vacío o nulo	Se rechaza la solicitud con un mensaje de validación.
PU03	Código único	Código ya registrado	Se rechaza la operación y no se agrega otro producto.
PU04	Precio válido	Precio negativo	Se rechaza la solicitud.
PU05	Stock inicial válido	Stock inicial negativo	Se rechaza la solicitud.
PU06	Stock mínimo válido	Stock mínimo negativo	Se rechaza la solicitud.
PU07	Categoría existente	Id de categoría inexistente o inactiva	Se rechaza la solicitud.
PU08	Permisos	Usuario sin permiso de inventario	Se bloquea la operación.
PU09	Alerta de stock	Stock inicial menor o igual al mínimo	Se crea producto y se genera una notificación.
PU10	Trazabilidad	Alta válida con stock inicial positivo	Se crea exactamente un movimiento de tipo Entrada.

Ejercicio 10. Pruebas de integración

Las pruebas de integración deben ejecutarse en un entorno donde la interfaz, el backend y la base de datos estén conectados. Como la evidencia depende del entorno del grupo, se detalla el procedimiento y el resultado esperado, sin afirmar ejecuciones no demostradas.

ID	Integración	Procedimiento	Resultado esperado
PI01	Formulario → API → BD	Registrar un producto válido desde el formulario.	El backend responde 201; el registro existe en Producto.
PI02	Producto + movimiento	Registrar producto con stock inicial 20.	Se guarda Producto y una Entrada de 20 en MovimientoInventario.
PI03	Validación de código	Enviar código duplicado por solicitud HTTP.	El backend responde 400; la interfaz muestra el error.

PI04	Categoría	Seleccionar una categoría activa.	El producto conserva la FK correcta y se muestra con la categoría elegida.
PI05	Alerta de stock	Crear producto con stock 2 y mínimo 5.	Se persiste la notificación y se visualiza en el panel.
PI06	Actualización de vista	Guardar producto válido.	La tabla y los indicadores del dashboard se actualizan sin recargar manualmente.

6. Despliegue del programa

Ejercicio 11. Plan de despliegue

Etapa	Acciones
1. Preparación	Congelar la versión aprobada, revisar requisitos de infraestructura, definir responsables y preparar un plan de reversión.
2. Respaldo	Realizar una copia de seguridad verificable de la base de datos y de la versión anterior de la aplicación.
3. Preparación del entorno	Configurar servidor web, backend, variables de entorno, credenciales seguras, motor de base de datos y certificado HTTPS.
4. Migración de datos	Ejecutar scripts DDL/DML para crear tablas, claves, restricciones e información inicial de roles/categorías.
5. Publicación	Transferir el paquete de la aplicación, configurar rutas y habilitar servicios de backend y frontend.
6. Pruebas de humo	Verificar inicio de sesión, alta de producto, registro de movimiento, consulta de alertas y acceso a reportes.
7. Aprobación y monitoreo	Obtener la conformidad de QA/cliente y monitorear logs, disponibilidad y errores durante las primeras horas.
8. Reversión	Ante una falla crítica, restaurar la versión anterior y la copia de seguridad siguiendo un procedimiento documentado.

Ejercicio 12. Despliegue propuesto en un servidor de producción

Para poner en producción la aplicación se recomienda utilizar un servidor con acceso HTTPS y una base de datos relacional protegida. La ejecución propuesta consiste en:

1. Publicar la versión aprobada del frontend y backend en el servidor de producción.
2. Configurar las variables de entorno: cadena de conexión, credenciales, dominio, puertos y claves de seguridad.
3. Ejecutar la migración de la base de datos y verificar claves foráneas, restricciones e índices.
4. Configurar el servidor web o proxy inverso para que entregue el frontend y redireccione las solicitudes a la API.
5. Instalar o renovar el certificado HTTPS.
6. Realizar una prueba de humo con un usuario autorizado y un producto de prueba.
7. Validar desde otro navegador o dispositivo que el producto, movimiento y alerta se registren correctamente.
8. Guardar capturas de la verificación y comunicar la disponibilidad a los usuarios finales.

7. Mantenimiento

Ejercicio 13. Plan de mantenimiento

Tipo / actividad	Acciones planificadas
Preventivo	Actualizar dependencias, revisar registros, verificar copias de seguridad y aplicar parches de seguridad.
Correctivo	Registrar incidencias, analizar causa raíz, corregir, probar y documentar cada solución.
Adaptativo	Ajustar la aplicación a cambios de navegadores, infraestructura, normas internas o necesidades operativas.
Evolutivo	Incorporar mejoras solicitadas: nuevos reportes, más filtros, proveedores o integración con sistemas externos.
Monitoreo	Supervisar disponibilidad, errores de aplicación, tiempos de consulta y capacidad de la base de datos.
Respaldo y recuperación	Ejecutar copias de seguridad automáticas y probar periódicamente la restauración de una copia.

Ejercicio 14. Corrección de error detectado

Incidencia propuesta: al registrar un producto con stock inicial, el producto se guarda correctamente pero, por una falla en la lógica, no siempre se registra el movimiento inicial. Esto ocasiona una inconsistencia entre el stock actual y el historial de movimientos.

1. Identificar la incidencia mediante un reporte de usuario o la revisión de logs.
2. Reproducir el error en un entorno de desarrollo con datos controlados.
3. Analizar la lógica de alta y confirmar que Producto y MovimientoInventario se guardan en operaciones separadas.
4. Corregir la implementación utilizando una transacción: si falla la creación del movimiento, se revierte también el alta del producto.
5. Agregar una prueba unitaria y una prueba de integración que verifiquen que todo producto nuevo con stock inicial genera un movimiento de Entrada.
6. Ejecutar la batería de pruebas completa para evitar efectos colaterales.
7. Desplegar la corrección en una ventana controlada, verificar logs y documentar la incidencia resuelta.

8. Nos preparamos para nuevos retos

Ejercicio 15. Equipo para un futuro dominio de IA generativa

Se propone que el futuro dominio de aplicación sea una plataforma de asistencia documental para pymes. La herramienta permitiría resumir documentos, responder preguntas sobre material interno y generar borradores controlados. El equipo requerido sería:

Rol	Responsabilidades principales
Gerente de proyecto	Coordina objetivos, tiempos, riesgos, presupuesto y comunicación con stakeholders.
Analista de requisitos / negocio	Define los casos de uso, restricciones, criterios de aceptación y necesidades reales de los usuarios.
Especialista UX/UI	Diseña una interacción clara, explica los límites de la IA y facilita la revisión humana de las respuestas.
Arquitecto de software	Define la arquitectura, integraciones con modelos generativos, seguridad, escalabilidad y trazabilidad.
Desarrollador frontend	Construye la interfaz, formularios, visualización de resultados y mecanismos de feedback.

Desarrollador backend	Implementa APIs, autenticación, reglas de negocio, almacenamiento y conexión con servicios de IA.
Ingeniero de datos / IA	Prepara datos, define estrategias de recuperación de información, evalúa calidad y reduce sesgos o alucinaciones.
QA / analista de calidad	Diseña pruebas funcionales, de seguridad, rendimiento y evaluación de respuestas generadas.
DevOps / infraestructura	Gestiona despliegue, observabilidad, copias de seguridad, escalamiento y costos de cómputo.
Responsable de seguridad y ética	Controla privacidad, permisos, uso responsable de datos y revisión de riesgos asociados al contenido generado.

Conclusión

La aplicación de inventario fue diseñada siguiendo una secuencia de trabajo en cascada: los requerimientos determinan el alcance; los diagramas y prototipos representan el diseño; la lógica de negocio define el comportamiento; las pruebas verifican el funcionamiento; y los planes de despliegue y mantenimiento aseguran continuidad operativa. La funcionalidad Agregar nuevo producto actúa como eje integrador porque conecta interfaz, validaciones, persistencia, movimientos de stock, alertas y pruebas.